

Lecture 10

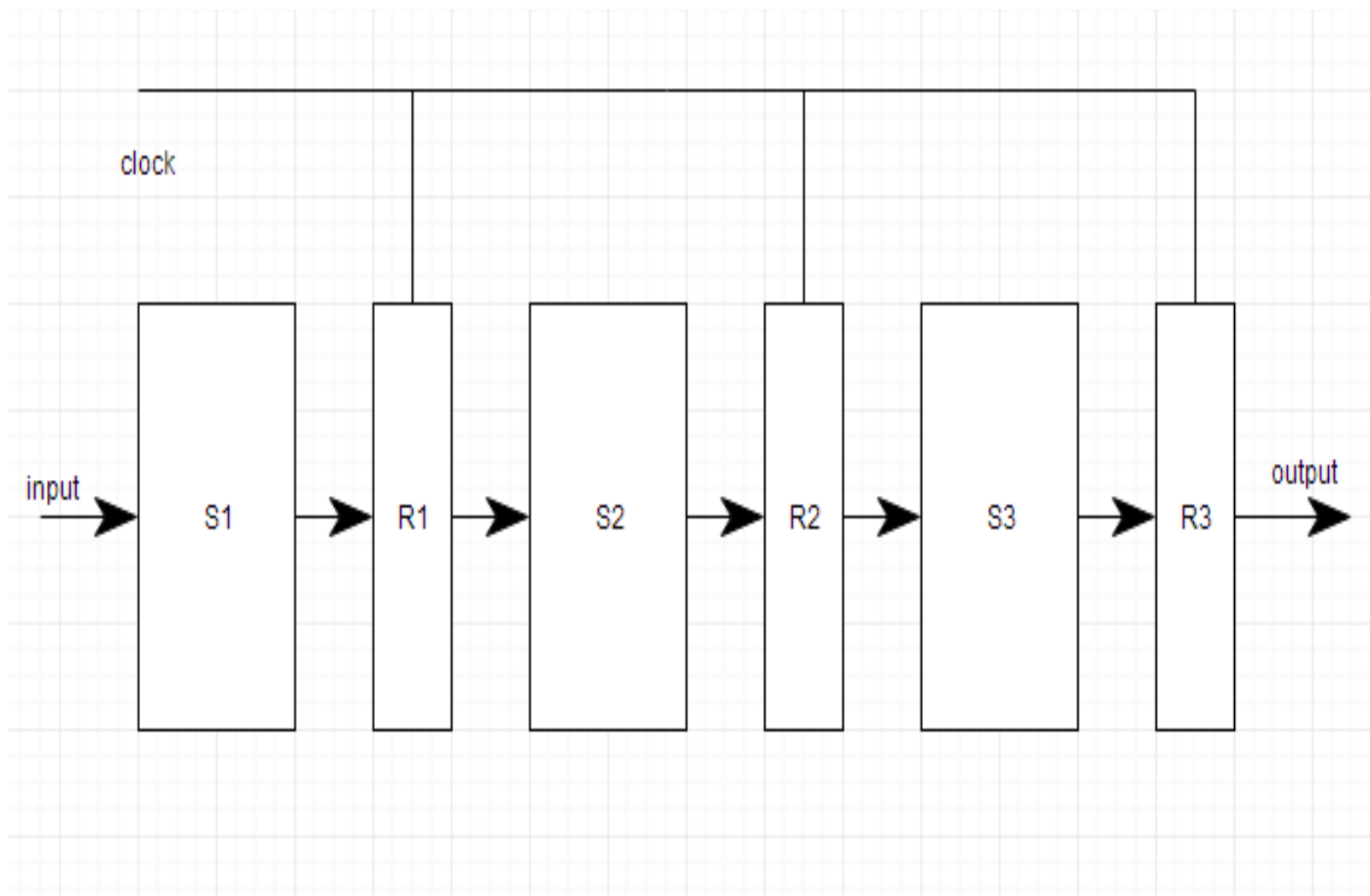
Pipelining: Basic Concepts

What Is Pipelining?

- *Pipelining* is an implementation technique whereby multiple instructions are overlapped in execution.
- If the system repeatedly executes a basic function, it divides the execution of an operation into a sequential independent stages.
- In a computer pipeline, each stage in the pipeline completes a part of an instruction.
- The stages are connected one to the next to form a pipe through a register for synchronization.

Cont.,

- Although it is critical to ensure that instructions in the pipeline do not attempt to use the hardware resources at the same time, we must also ensure that instructions in different stages of the pipeline do not interfere with one another.
- This separation is done by introducing *pipeline registers* between successive stages of the pipeline, so that at the end of a clock cycle all the results from a given stage are stored into a register that is used as the input to the next stage on the next clock cycle.



Optimization of a Pipeline

- Because the pipe stages are hooked together, all the stages must be ready to proceed at the same time.
- Because all stages proceed at the same time, the length of a processor cycle is determined by the time required for the slowest pipe stage.
- The pipeline designer's goal is to balance the length of each pipeline stage.

Speedup

- The throughput of an instruction pipeline is determined by how often an instruction exits the pipeline.
- Let a pipeline of depth k , can process n items in ' $n + k$ ' steps (throughput), where, without pipeline it takes ' $n \times k$ ', therefore, $\text{Speedup} = (n \times k) / (n + k)$

Example:

Add 100 pairs of numbers, one Add takes 5 cycles, i. e., $k = 5$, $n = 100$.

$$\text{Speedup} = 5$$

Major Areas in Computer Design

- Pipelines are used in two major areas in computer design:
 - Instruction Level Processing
 - Arithmetic Operations

Which yields, what we called 'Reduced Instruction Set Computer (RISC)'.

The Basics of a RISC Instruction Set

- All operations on data apply to data in registers and typically change the entire register (32 or 64 bits per register).
- The only operations that affect memory are load and store operations.
- The instruction formats are few in number, with all instructions typically being one size.

Classes of Instructions:

Most RISC architectures, like MIPS, have three classes of instructions:

1- *ALU instructions*—These instructions take either two registers or a register and a sign-extended immediate.

2- *Load and store instructions*

3- *Branches and jumps*

A Simple Implementation of a RISC Instruction Set

- To understand how a RISC instruction set can be implemented in a pipelined fashion, we need to understand how it is implemented *without* pipelining.
- Every instruction in this RISC subset can be implemented in at most 5 clock cycles.
- The 5 clock cycles are as follows:

IF? ID/ OF? EX? MEM? WB

Instruction Level Pipeline

- We can pipeline the execution described above with almost no changes by simply starting a new instruction on each clock cycle.
- Each of the clock cycles from the previous section becomes a *pipe stage*—a cycle in the pipeline.
- Although each instruction takes 5 clock cycles to complete, during each clock cycle the hardware will initiate a new instruction and will be executing some part of the five different instructions.

Clock number									
Instruction number	1	2	3	4	5	6	7	8	9
Instruction i	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Problems that pipelining introduces

- To start with, we have to determine what happens on every clock cycle of the processor and make sure we don't try to perform two different operations with the same data path resource on the same clock cycle.
- For example, a single ALU cannot be asked to compute an effective address (for memory operation) and perform a subtract operation at the same time.

Cont.,

- We use separate instruction and data memories, which we would typically implement with separate instruction and data caches to eliminate a conflict for a single memory that would arise between instruction fetch and data memory access.
- The register file is used in the two stages: one for reading in ID and one for writing in WB. To handle reads and a write to the same register, we perform the register write in the first half of the clock cycle and the read in the second half.

Cont.,

- To start a new instruction every clock, we must increment and store the PC every clock, and this must be done during the IF stage in preparation for the next instruction.
- Furthermore, we must also have an adder to compute the potential branch target during ID.
- One further problem is that a branch does not change the PC until the ID stage.

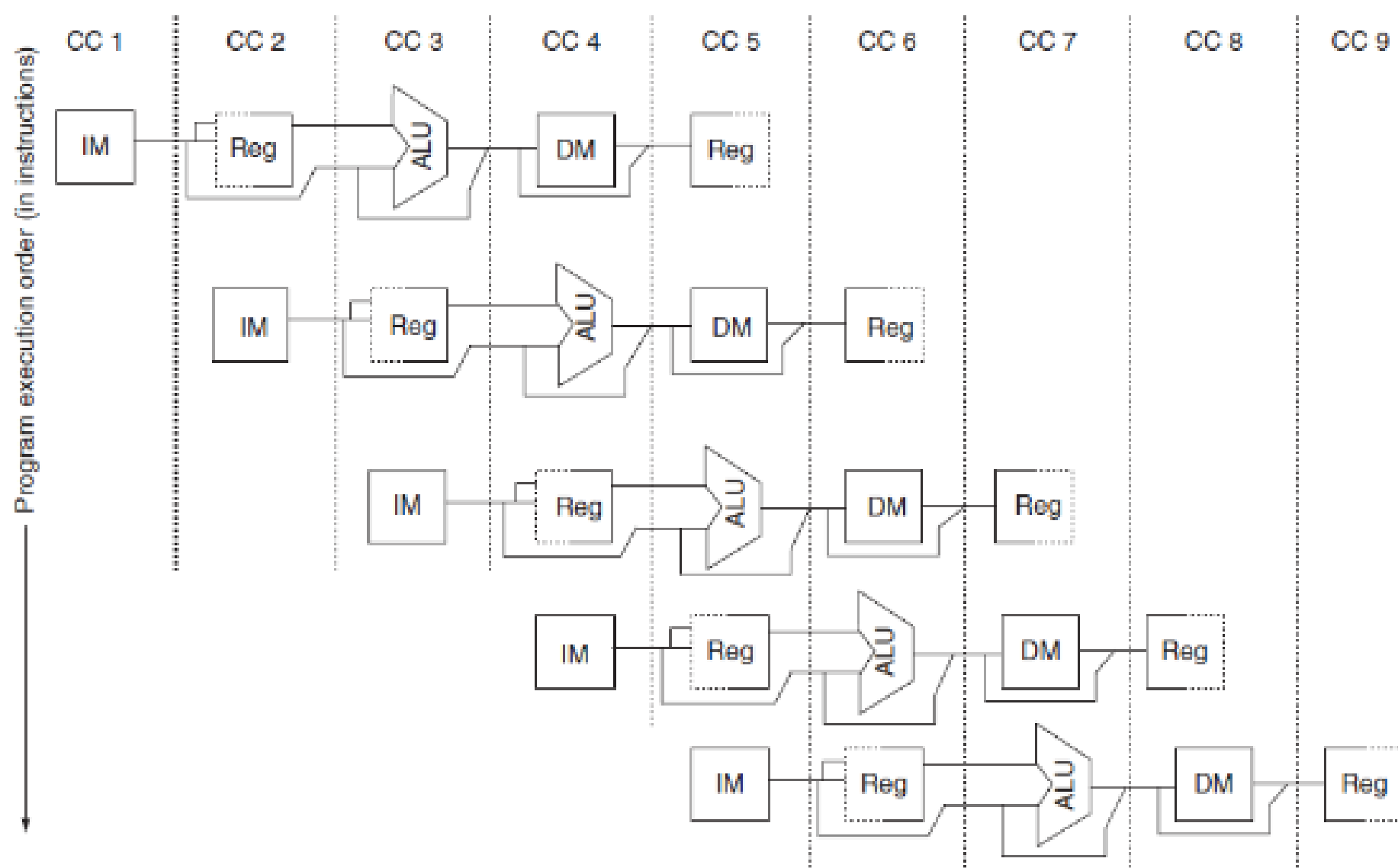
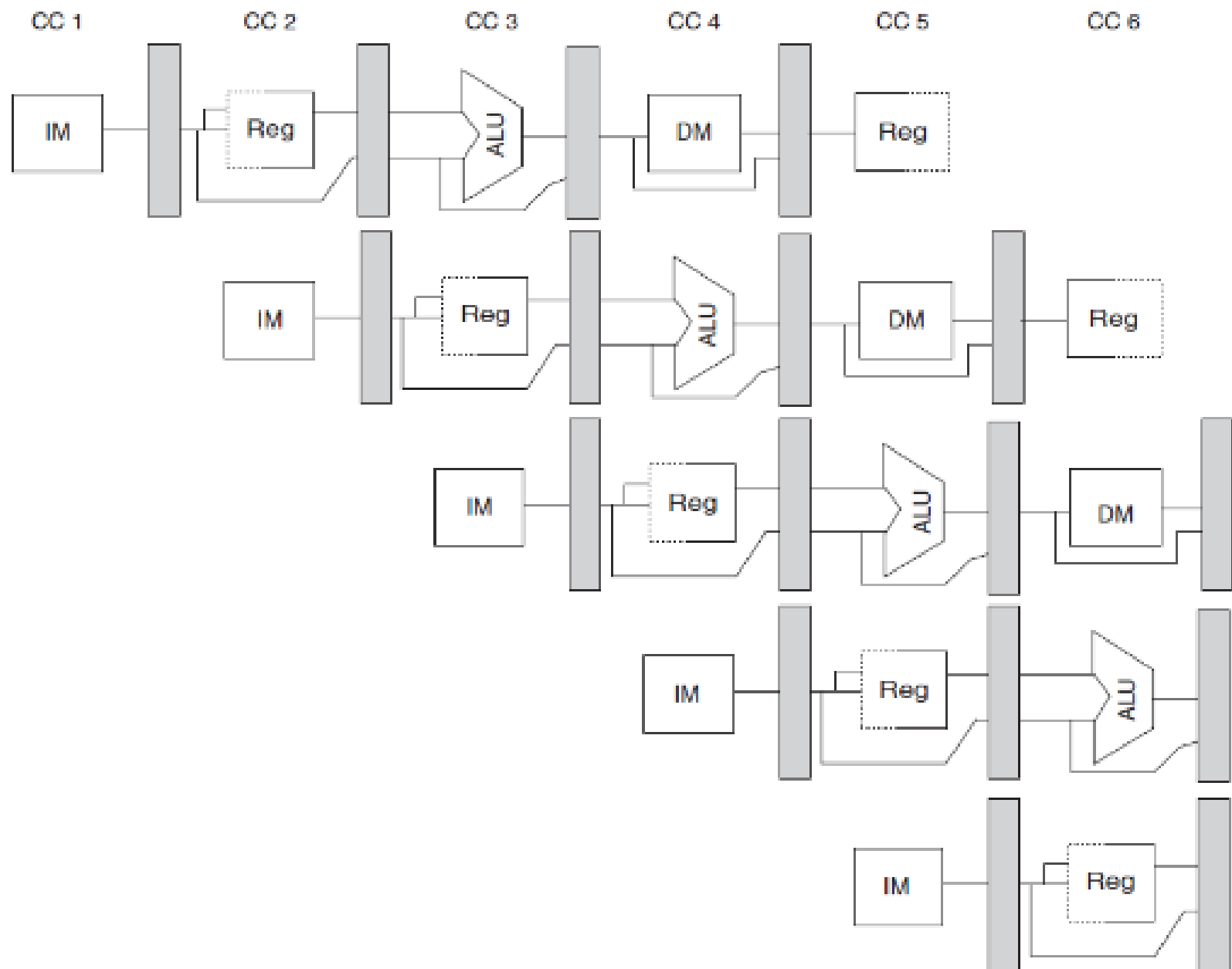


Figure C.2 The pipeline can be thought of as a series of data paths shifted in time. This shows the overlap among the parts of the data path, with clock cycle 5 (CC 5) showing the steady-state situation. Because the register file is used as a source in the ID stage and as a destination in the WB stage, it appears twice. We show that it is read in one part of the stage and written in another by using a solid line, on the right or left, respectively, and a dashed line on the other side. The abbreviation IM is used for instruction memory, DM for data memory, and CC for clock cycle.

Pipeline Registers

- Although it is critical to ensure that instructions in the pipeline do not attempt to use the hardware resources at the same time, and also ensure that instructions in different stages of the pipeline do not interfere with one another.
- This separation is done by introducing *pipeline registers* between successive stages of the pipeline.
- So that at the end of a clock cycle all the results from a given stage are stored into a register that is used as the input to the next stage on the next clock cycle.

Time (in clock cycles) →



Pipeline Hazards

- There are situations, called *hazards*, that prevent the next instruction in the instruction stream from executing during its designated clock cycle.
- Hazards reduce the performance from the ideal speedup gained by pipelining.

Classes of hazards

- There are three classes of hazards:
- 1. *Structural hazards* arise from resource conflicts when the hardware cannot support all possible combinations of instructions simultaneously in overlapped execution.
- 2. *Data hazards* arise when an instruction depends on the results of a previous instruction .
- 3. *Control hazards* arise from the pipelining of branches and other instructions that change the PC.

Cont.,

- Hazards in pipelines can make it necessary to *stall* the pipeline.
- Instructions issued *earlier* than the stalled instruction—and hence farther along in the pipeline—must continue, since otherwise the hazard will never clear.
- As a result, no new instructions are fetched during the stall.

Solution for Structural Hazards

- When a processor is pipelined, the overlapped execution of instructions requires pipelining of functional units and **duplication of resources** to allow all possible combinations of instructions in the pipeline.
- Structural hazards arise when some functional unit is not fully pipelined.
- Another common way that structural hazards appear is when some resource has not been duplicated enough to allow all combinations of instructions in the pipeline to execute.

Example: a load with one memory port

- Some pipelined processors have shared a single-memory pipeline for data and instructions.
- As a result, when an instruction contains a data memory reference, it will conflict with the instruction reference for a later instruction, as shown in [Figure C.4](#).
- To resolve this hazard, we stall the pipeline for 1 clock cycle when the data memory access occurs.
- For the following figure, the load instruction uses the memory for a data access, and at the same time instruction 3 wants to fetch an instruction from memory.

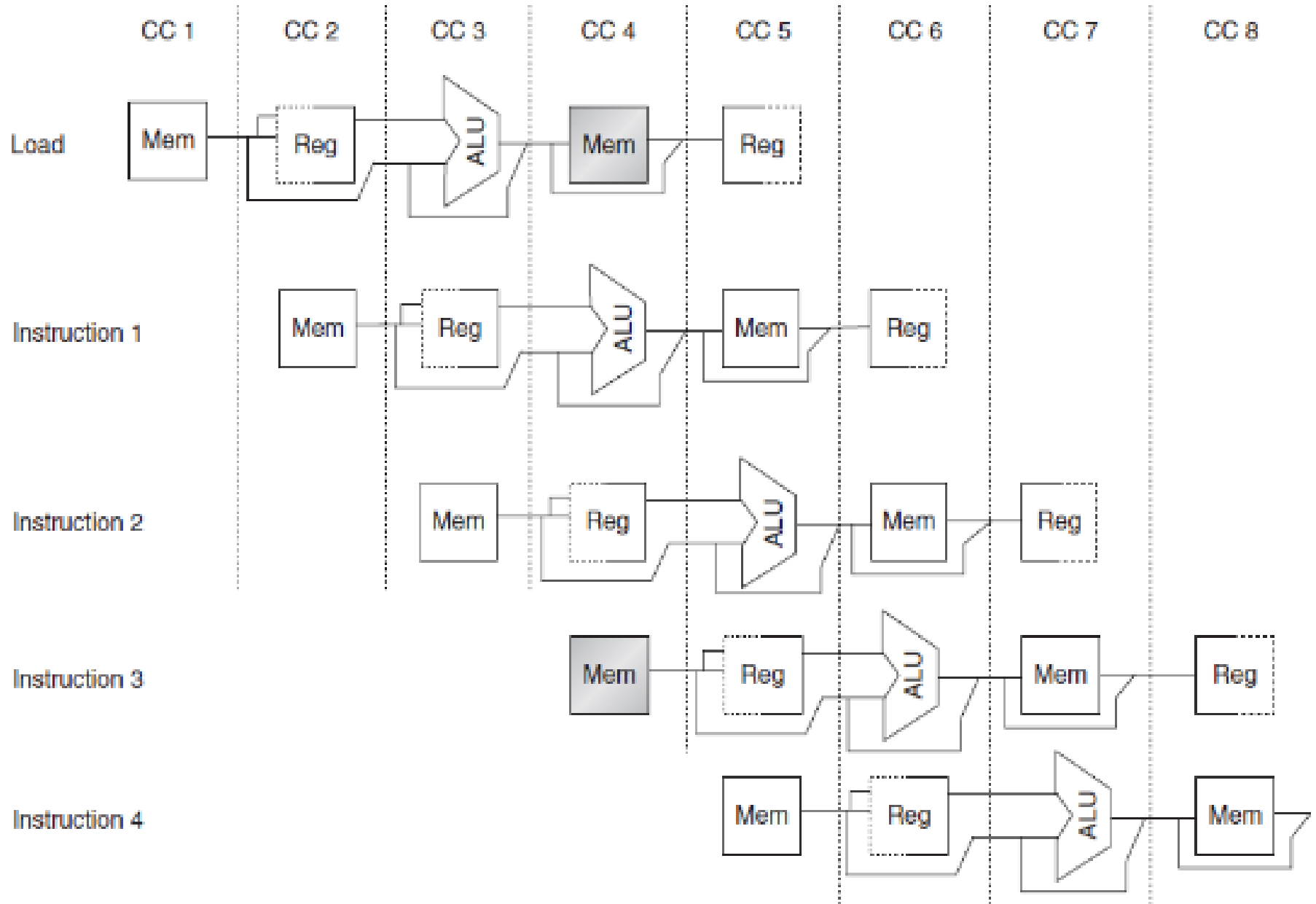


Figure C.4 A processor with only one memory port will generate a conflict whenever a memory reference occurs. In this example the load instruction uses the memory for a data access at the same time instruction 3 wants to fetch an instruction from memory.

	Clock cycle number									
Instruction	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i + 1$		IF	ID	EX	MEM	WB				
Instruction $i + 2$			IF	ID	EX	MEM	WB			
Instruction $i + 3$				Stall	IF	ID	EX	MEM	WB	
Instruction $i + 4$						IF	ID	EX	MEM	WB
Instruction $i + 5$							IF	ID	EX	MEM
Instruction $i + 6$								IF	ID	EX

Data Hazards

Consider the pipelined execution of these instructions:

I1: ADD R1,R2,R3

I2: SUB R4,R1,R5

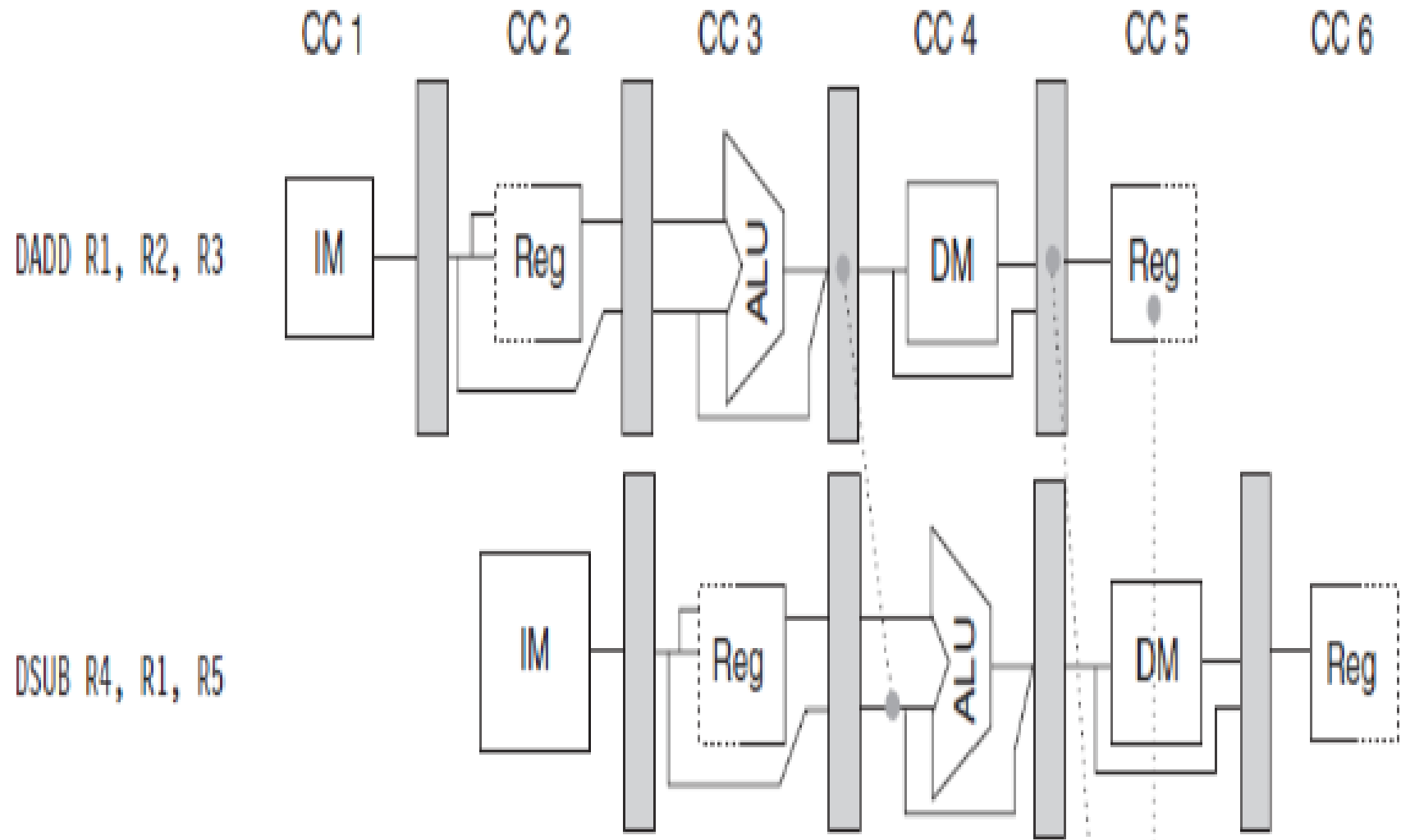
Where, R1 is the output of I1, and input in I2.

Solution:

By using Forwarding (Bypassing) hardware.

When it detects that the previous ALU operation has the source for the current ALU operation, control logic (MUX) selects the forwarded result as an ALU input rather than the value read from the register file.

Time (in clock cycles) →



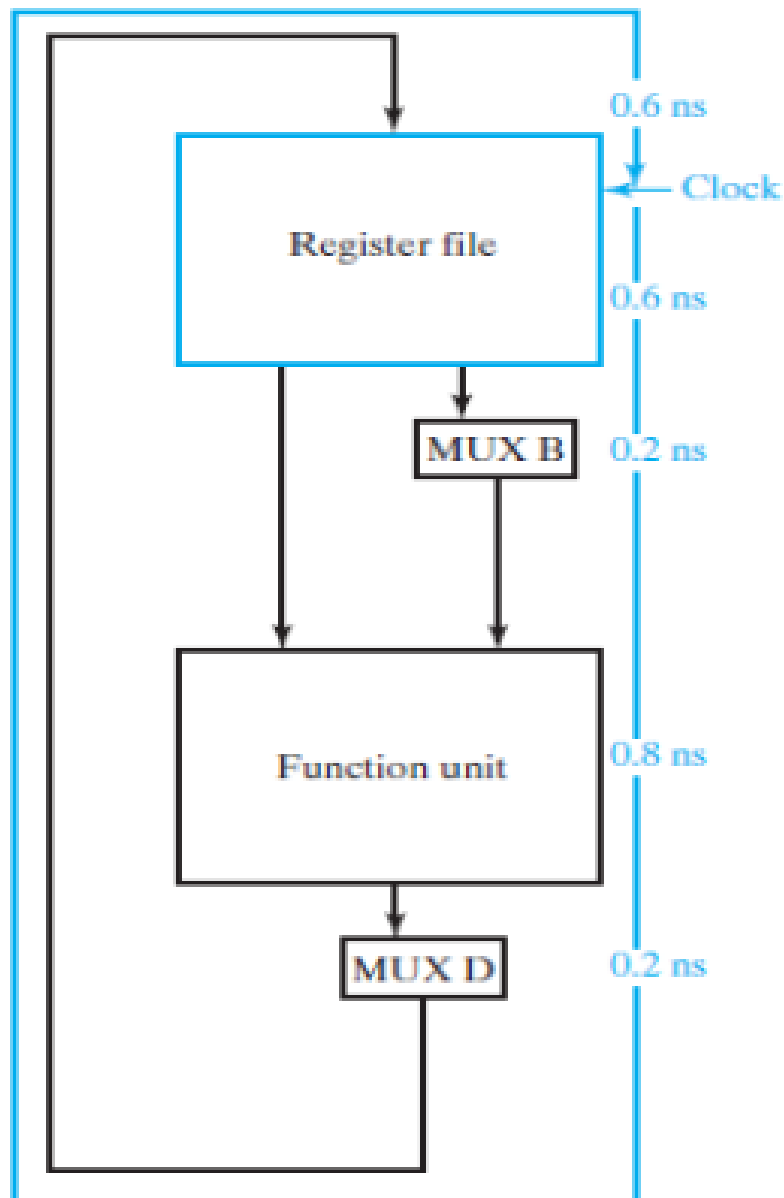
Branch Hazards (sol: Branch Prediction)

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

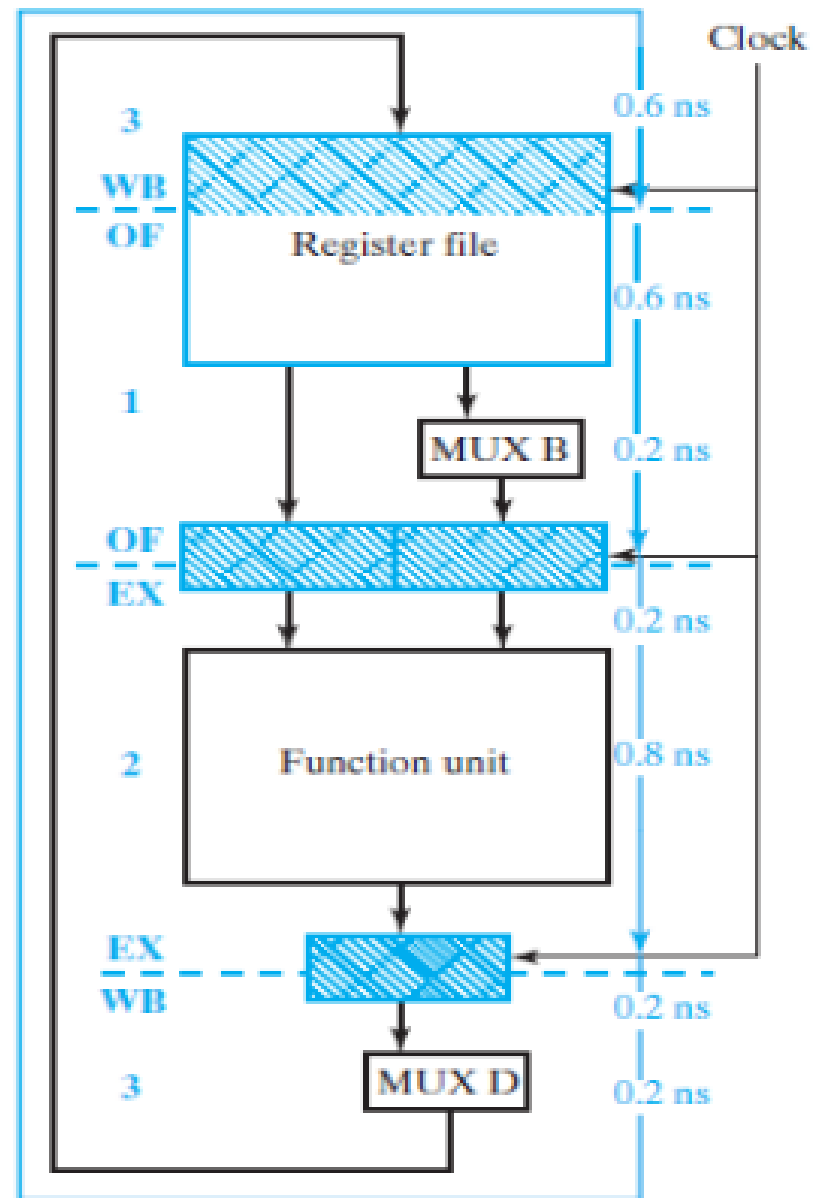
Figure C.11 A branch causes a one-cycle stall in the five-stage pipeline. The instruction after the branch is fetched, but the instruction is ignored, and the fetch is restarted once the branch target is known. It is probably obvious that if the branch is not taken, the second IF for branch successor is redundant. This will be addressed shortly.

Pipelined Datapath

- The following Figure (a) illustrates maximum delay values for each of the components of a typical datapath.
- We find that 2.4 ns is required to perform a single microoperation.
- The resulting datapath, sketched in Figure (b), is referred to as a *pipelined datapath*, or just a *pipeline*.



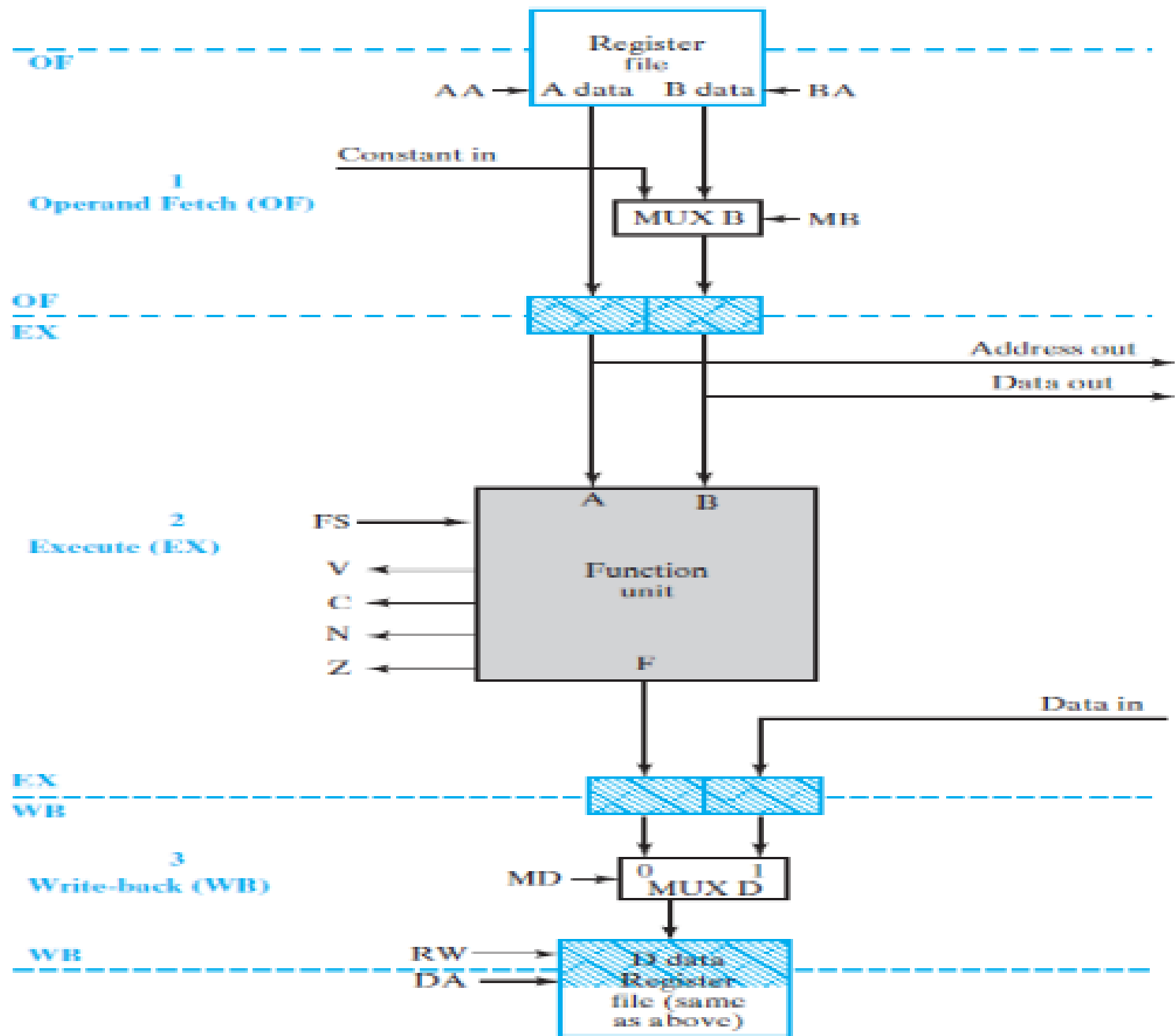
(a) Conventional



(b) Pipelined

Cont.,

- The first stage, OF, is the operand fetch stage. The operand fetch consists of reading register values to be used from the register file and, for Bus B, selecting between a register value or a constant by using MUX *B*.
- Following the OF stage is the first pipeline platform.
- The pipeline registers store the operand or operands for use in the next stage during the next clock cycle.
- And so on for the next stages.



Pipelined Control

- The control has an added stage for instruction fetch that includes the *PC* and instruction memory.
- This becomes stage 1 of the combined pipeline.
- The instruction decoder and register file read are now in stage 2.
- The function unit and data memory read and write are in stage 3,
- And the register file write back is in stage 4.
- The control information held in the final pipeline platform consists of DA, MD, and RW, which are used in the final write-back stage, WB.

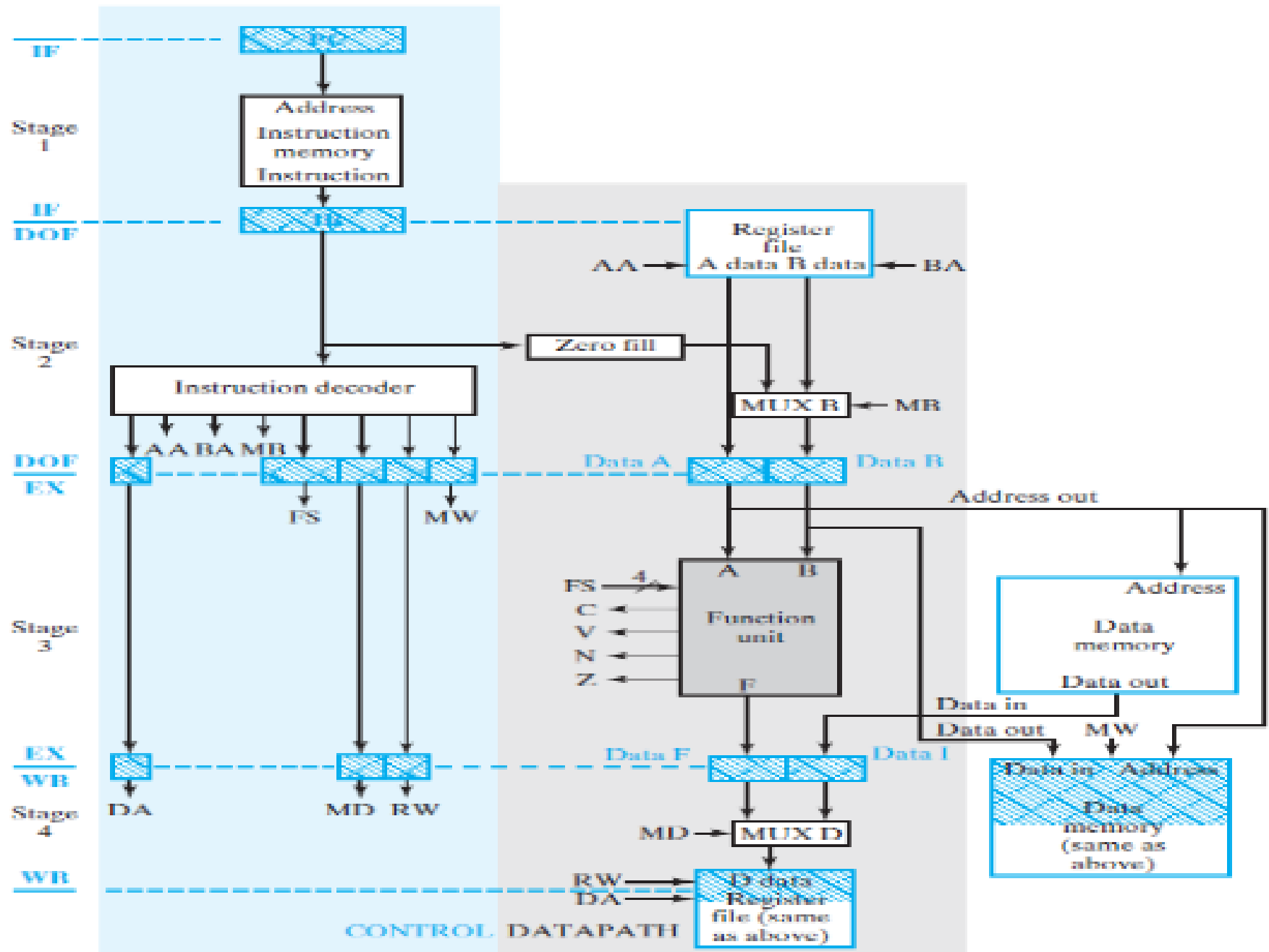


FIGURE 10-4
Block Diagram of Pipelined Computer

Sheet

- Text Book 'Hennessy